1. SOLUTION DE L'EXERCICE CALCUL DU SALAIRE MOYEN

A. Création de la fonction :

```
CREATE OR REPLACE FUNCTION moySalaire() RETURNS NUMERIC AS $$
DECLARE
    V_Moyenne numeric;
BEGIN
    SELECT INTO V_Moyenne AVG(salaire) FROM employes;
    RETURN V_Moyenne;
END
$$ LANGUAGE plpgsql;
```

B. Un exemple d'utilisation « en direct » :

```
SELECT moySalaire();
```

C. Les noms et le salaire des employés qui gagnent plus que le salaire moyen :

```
SELECT nom,salaire
FROM employes
WHERE salaire > moySalaire();
```

D. Ceux dont le salaire est égal au salaire moyen à 10% près :

```
SELECT nom,salaire
FROM employes
WHERE ABS(salaire - moySalaire())<0.10 * moySalaire();
```

Autre méthode

```
SELECT nom,salaire
FROM employes
WHERE salaire between (0.9 * moySalaire()) AND (1.1 * moySalaire());
```

2. SOLUTION DE L'EXERCICE RECHERCHE NOM DEPARTEMENT

E. Création de la fonction :

```
CREATE OR REPLACE FUNCTION departement(P_NumEmp  integer) RETURNS VARCHAR AS
$$
DECLARE
   V_dpt varchar;
BEGIN
       -- on peut vérifier que le N° de l'employé existe
       -- perform est comme le select mais ne sert qu'à évaluer une requête
       perform * from employes where noemp= P_NumEmp;
       -- si aucune ligne retournée
       IF not found
          THEN RAISE EXCEPTION 'le N° % d''employé n''existe pas' , P_NumEmp ;
       END IF;

    SELECT d.nom INTO V_dpt FROM employes e, departement d
         WHERE e.nodept=d.nodept AND noemp= P_NumEmp ;
    RETURN V_dpt;
END;
$$ LANGUAGE plpgsql;
```

Un exemple d'utilisation en "direct" :

```
SELECT departement(2); -- sans erreur
SELECT departement(255); -- avec erreur sur le N° de l'employé
```

3. SOLUTION DE L'EXERCICE RETOURNER LES COLLEGUES

```
-- Première solution SANS CURSEUR
CREATE OR REPLACE FUNCTION collegues(P_NumEmp integer) RETURNS SETOF RECORD  AS $$
 DECLARE
    V_rec RECORD;
 BEGIN
    -- construction du tableau de résultat
    -- on utilise la fonction créé en 2 qui nous retourne le département
    -- d'un employé passé en paramètre

    FOR V_rec IN SELECT prenom ,e.nom
             FROM employes e inner join departement  d on e.nodept=d.nodept
             WHERE d.nom= departement(P_NumEmp)
             AND noemp != P_NumEmp
        -- retour des valeurs du tableau V_rec
        LOOP
             RETURN NEXT V_rec ;
        END LOOP ;

    RETURN ;
 END;
$$LANGUAGE plpgsql;
-------------------------------------------------------------------------
-- deuxième solution AVEC CURSEUR

CREATE OR REPLACE FUNCTION collegues_curs(P_NumEmp integer) RETURNS SETOF RECORD  AS
$$
 DECLARE
    V_rec RECORD;
    curs1 CURSOR(key integer) IS SELECT prenom ,e.nom
                    FROM employes e inner join departement  d on e.nodept=d.nodept
                    WHERE d.nom= departement(key)
                    AND noemp != key;

 BEGIN
    open curs1(P_NumEmp); -- ouverture du curseur
    LOOP
        FETCH curs1 INTO  V_rec;
        EXIT WHEN NOT FOUND;
        RETURN NEXT V_rec ;
    END LOOP ;
    RETURN ;
 END;
$$LANGUAGE plpgsql;
```

Exemple d'utilisation :

```
SELECT * FROM collegues(7) AS (prenom VARCHAR,nom VARCHAR);

-- Avec le curseur
SELECT * FROM collegues_curs(7) AS (prenom VARCHAR,nom VARCHAR);
```

4. SOLUTION DE L'EXERCICE : NOM ET PRENOM DES SUPERIEURS

Fonction qui retourne LE supérieur de l'employé passé en paramètre

```
CREATE OR REPLACE FUNCTION sup(P_NumEmp  integer) RETURNS RECORD AS $$
DECLARE
       Var_NomSup RECORD;

BEGIN
    -- valorisation du record avec le résultat de la requête
      SELECT nom ,prenom, nosupr INTO Var_NomSup from employes
                  WHERE noemp=(SELECT nosupr FROM employes  WHERE noemp= P_NumEmp);
      RETURN Var_NomSup ;
END;
 $$LANGUAGE plpgsql;
```

On la teste ainsi :

```
SELECT * FROM sup(17)
     AS (nom VARCHAR,prenom VARCHAR,nosupr NUMERIC(7,0));
```

Fonction qui retourne LES supérieurs de l'employé passé en paramètre

Cette fonction utilise une table temporaire,
en page suivante vous avez une autre façon de faire sans table temporaire.

```
CREATE OR REPLACE FUNCTION superieurs_temp(p_numEmp integer) RETURNS SETOF RECORD AS
$$
 DECLARE
    rec RECORD;
    rec1 RECORD;
    superior INTEGER;
    save integer;
 BEGIN
-- creation d'une table temporaire
--
    create table tempsup ( nom VARCHAR , prenom VARCHAR,  nosupr NUMERIC(7,0));

    save:= p_numEmp; -- initialise  avec  le paramétre
    LOOP
        -- vérifie qu'il existe un supérieur pour cet employé,
        -- si il n'y a plus de supérieur on sort de la boucle
        SELECT nosupr INTO superior FROM employes WHERE noemp= save;
        EXIT WHEN superior IS NULL;

        -- sinon on sélectionne la ligne du supérieur (fonction sup)
        -- est on l'insere dans la table temporaire
        SELECT into rec * FROM sup(save)
                AS (nom VARCHAR,prenom VARCHAR,nosupr NUMERIC(7,0));
        INSERT INTO tempsup VALUES(rec.nom,rec.prenom,rec.nosupr);
        -- recup du num de supérieur courant
        save:=superior;

    END LOOP;
    -- lit la table temporaire
    FOR rec1 in SELECT * FROM tempsup
    LOOP RETURN NEXT rec1 ;
    END LOOP;
    DROP table tempsup;
    RETURN;
 END; $$LANGUAGE plpgsql;
```

On la teste ainsi :

```
SELECT * FROM superieurs_temp(17)
     AS (nom VARCHAR,prenom VARCHAR,nosupr NUMERIC(7,0));
```

Fonction qui retourne LES supérieurs de l'employé passé en paramètre

Sans l'utilisation de table temporaire

```
CREATE OR REPLACE FUNCTION superieurs(p_numEmp integer) RETURNS SETOF RECORD AS $$
DECLARE
   V_rec_sup RECORD;
   superior INTEGER;
   save integer;
BEGIN

    -- on peut vérifier que le N° de l'employé existe
    -- perform est comme le select mais ne sert qu'à évaluer une requête
    perform * from employes where noemp= P_NumEmp;
    -- si aucune ligne retournée
    IF not found
         THEN RAISE EXCEPTION 'le N° % d''employé n''existe pas' , P_NumEmp ;
    END IF;

    save:= p_numEmp;
    LOOP
       -- vérifie qu''il existe un supérieur pour l''employé courant,
       -- sinon on sort de la boucle
       SELECT nosupr INTO superior FROM employes WHERE noemp= save;
       EXIT WHEN superior IS NULL;

       -- SI OUI on récupère les infos du supérieur
       SELECT into V_rec_sup * FROM sup(save)
                    AS (nom VARCHAR,prenom VARCHAR,nosupr NUMERIC(7,0));

       -- retourne le record
       RETURN NEXT V_rec_sup;

       -- recup du numéro du supérieur courant
       save:=superior;

       END LOOP;
   RETURN;
END;
$$LANGUAGE plpgsql;
```

```
SELECT * FROM superieurs(17)
 AS (nom VARCHAR, prenom VARCHAR, nosupr NUMERIC); -- sans erreur

SELECT * FROM superieurs(78)
 AS (nom VARCHAR, prenom VARCHAR, nosupr NUMERIC); -- avec erreur de N° d'employé
```